

LogicConnect: Structured Connectivity Design for Differentiable Logic Gate Networks

Abstract—Logic gate networks learn Boolean functions on a fixed graph, making connectivity a design choice that directly constrains the signals available to each gate. We present LogicConnect, a methodology for constructing this graph in dense and convolutional logic gate networks. The method combines semantic input pairing with deterministic multiscale matching stages, prioritizing balanced predecessor use and using ancestry overlap to select among equally balanced stages. Construction occurs before training and adds neither trainable routing parameters nor inference operators. Experiments on MNIST, Fashion-MNIST, and CIFAR-10 measure hardened accuracy and training cost under matched architectural budgets. On dense CIFAR-10 models, the unified construction improves mean test accuracy by 3.18-4.59 percentage points across three paired seeds. Convolutional S and M models gain 3.26 and 2.08 points in single-seed tests, although additional S replications yield an inconclusive three-seed validation improvement. Degree-preserving ablations identify first-layer structure as the dominant contributor in the evaluated dense model. The results support offline connectivity design as a practical accuracy and training-resource trade-off.

I. INTRODUCTION

In a logic gate network (LGN), each two-input gate combines only the signals delivered by its two predecessors. These connections therefore determine which input combinations the model can represent within a fixed depth and gate budget. Differentiable training selects gate functions but cannot recover a missing connection in a fixed graph, making connectivity a design opportunity before gate-function optimization begins.

Existing LGNs expose two ways to make this choice. Random fixed connections require no routing optimization and produce ordinary Boolean circuits after training [1]. Learned connections search over candidate predecessors and can achieve higher accuracy with compact networks [2], [3]. However, training the connections requires additional routing state and candidate-signal processing, increasing the demand on limited GPU memory. A designer facing this constraint needs to understand the accuracy available from a structured fixed graph before accepting that additional cost.

This choice becomes more involved when the same design principle must support dense and convolutional LGNs. A dense input contains spatial coordinates, channels, and potentially several thermometer thresholds per raw value. A convolutional network instead shares logic trees across spatial positions and selects channels within each receptive field [4]. A unified

method must preserve these architectural semantics while organizing the predecessors accessible to each gate throughout the network.

We address this problem with LogicConnect (U2), an offline construction that pairs semantically related inputs and organizes deeper connections through deterministic matching stages. Stage selection first minimizes predecessor fan-out spread, then uses ancestry overlap to choose among equally balanced candidates. The same construction operates on dense predecessors and convolutional channel groups, while retaining the established convolutional spatial sampler. Only the resulting connection indices enter the trained and deployed model.

Our contributions are threefold:

- We formulate a shared connectivity construction for dense layers, convolutional channel groups, and the convolutional classifier. The method preserves the gate budget and introduces no trainable routing parameters or additional inference operators.
- We evaluate hardened accuracy and training resources using paired random-connectivity controls and explicit learned-routing comparisons. Dense CIFAR-10 improvements are consistent across three model sizes; convolutional results expose a more variable benefit that we report separately.
- We isolate first-layer and deeper-layer structure using randomizations that preserve each predecessor’s degree in each input slot. The resulting dense ablation identifies a strong first-layer contribution, while leaving the incremental deeper-layer benefit uncertain.

These experiments support structured connectivity as an architectural option, with comparisons against the stronger dense specialization V3 and configurations where learned routing remains more accurate. To make that distinction precise, the following section separates connectivity design from gate parameterization and circuit simplification before presenting the unified construction.

II. RELATED WORK

Differentiable logic architectures. Petersen et al. train LGNs through continuous relaxations of Boolean functions, then discretize each gate for inference [1]. The connection graph remains independent of the learned gate functions, separating graph construction from the subsequent optimization of Boolean operations. Convolutional LGNs extend this architecture with shared logic trees, OR pooling, and residual initialization [4]. These architectural changes improve spatial processing while retaining a separate choice of channel connectivity that determines access within receptive fields. We study

this choice under matched architectural budgets, preserving the corresponding dense and convolutional operators throughout each paired comparison.

Learning the connections. Mommen et al. optimize distributions over subsets of candidate inputs to improve the connections available to each gate [2]. LILogic Net extends learnable connectivity with a sparse Top-K mechanism and efficient evaluation through basis projection [3]. These approaches address the limitations of fixed random graphs by training routing decisions, whereas our construction determines the graph before optimization. The relevant comparison includes both accuracy and training resources, since additional candidate signals can consume memory even when the deployed graph remains compact.

Separating the design axes. BitLogic makes this comparison broader by organizing LUT-native networks along axes that include encoding, connectivity, fan-in, and training choices [5]. Since a gain obtained by changing several axes cannot identify the effect of connectivity alone, we use matched random/U2 pairs for attribution. Comparisons involving different architectures or routing mechanisms retain explicit labels so that their accuracy gains do not imply a topology-only effect. Published accuracies provide context, while locally measured accuracy, memory, and wall time establish the trade-off under our execution conditions.

Gate parameterization. Light DLGNs and WARP change the representation used to train gate functions [6], [7]. Such representations affect parameter count, optimization, and hardening, addressing a different design variable from the predecessor indices studied here. Nevertheless, the resulting optimization behavior can interact with connectivity, making compatibility a question that requires evaluation under each gate representation. Our adapted WARP controls therefore test compatibility explicitly instead of assuming that a positive result with the original gate parameterization transfers to every alternative.

Simplifying trained circuits. Two-stage unit tying reduces the logical footprint of a trained LGN through functional approximation and subsequent refinement [8]. Unit tying changes the retained structure after training, whereas U2 constructs a graph at a declared budget before training. Both choices can matter to a deployment flow, but evaluating their joint benefit requires measurements beyond nominal gate count, including physical area and energy.

Together, these studies motivate a comparison that keeps the graph, the gate-training rule, and the deployment representation distinct. Our contribution is a shared fixed-connectivity construction with measured accuracy/resource trade-offs and controlled structural ablations that preserve local predecessor degrees. The following methodology defines the information used during construction, the pair-selection rule, and the indices retained by the trained model.

III. UNIFIED CONNECTIVITY CONSTRUCTION

A. Graph and construction metadata

We represent layer l by m_l two-input gates and a fixed predecessor matrix $I_l \in \{0, \dots, n_l - 1\}^{2 \times m_l}$. Here, n_l is the

preceding layer's width, and gate j receives distinct predecessors $a = I_l[0, j]$ and $b = I_l[1, j]$. During training, the gate uses the usual mixture of the 16 relaxed Boolean functions [1]:

$$h_{l,j} = \sum_{q=0}^{15} \text{softmax}(\theta_{l,j})_q \tilde{f}_q(h_{l-1,a}, h_{l-1,b}). \quad (1)$$

Training optimizes θ while keeping I_l fixed; hardening then selects the highest-weight Boolean function, and class-group summation produces the output scores. Our method changes only the construction of I_l ; the loss, gate parameterization, and inference operators remain those of the corresponding baseline.

To construct these indices, we associate each predecessor i with a set A_i of structural ancestry labels. In a dense image network, an input label identifies one raw channel-pixel value, so multiple thermometer thresholds of that value share a label. Propagation takes the union $A_j = A_a \cup A_b$, recording potential structural access without claiming functional dependence after gate learning. Figure 1 shows how the metadata supports construction and is then discarded.

B. Semantic input pairing

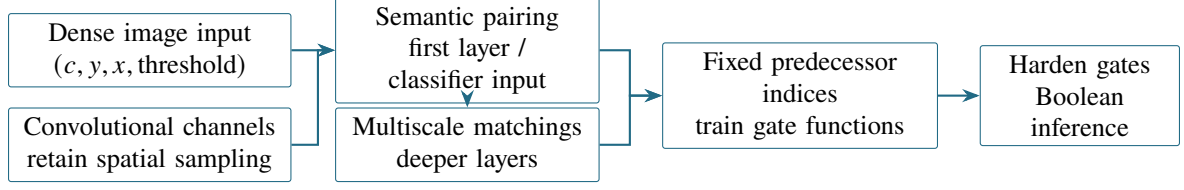
At the image input, ancestry alone does not distinguish useful pairings, so we retain each bit's channel, spatial coordinates, and threshold index. Candidate stages pair positions along the horizontal and vertical axes at increasing power-of-two strides, capped at half the axis length. The finest scale also includes aligned channel pairs and threshold-changing pairs with a spatial displacement, excluding pairs from the same raw source.

The constructor cycles through these semantic stages to represent several axes rather than exhausting one axis first. Within the selected stage, it examines up to 64 unused candidates and chooses the pair minimizing endpoint fan-out sum, then maximum endpoint fan-out, then endpoint indices. A seeded affine permutation determines candidate order without consuming the training generator. Once a stage's distinct pairs are exhausted, repeated pairs become eligible using the same degree ordering over the stage. This procedure provides a reproducible semantic input graph without claiming globally optimal balance.

C. Deeper multiscale matchings

For deeper layers, we replace semantic-coordinate stages with regular matchings over predecessor indices, using $K = \lceil \log_2 n \rceil$ scales for n inputs. At power-of-two widths, scale s pairs i with $i \oplus 2^s$. Otherwise, an affine traversal with a coprime step visits each predecessor once, and adjacent elements form pairs. Odd widths leave one predecessor unmatched, with the omitted index rotating across the schedule. For example, eight predecessors yield four disjoint pairs at each of three XOR scales. Filling ten gates uses two complete matchings and two pairs from a third, giving predecessor degrees of two or three.

These matchings control endpoint reuse independently of scale selection: d_i denotes current fan-out, and P_s contains pairs retained from candidate stage s . If only part of a stage fits the remaining width, we select its disjoint pairs by endpoint



Stage priority: degree spread $\rightarrow$ ancestry novelty $\rightarrow$ nominal scale

Fig. 1: One offline construction, two architecture adapters. Semantic pairing defines the input graph; matching stages organize deeper predecessors. Convolution retains the existing spatial sampler and shared logic trees. Construction metadata is discarded before deployment.

Algorithm 1 Deeper-layer stage selection

Require: Ancestry A , width m , layer l , seed
Initialize degrees $d = 0$, output pairs $I = \emptyset$.
while $|I| < m$ **do**
 Generate the K seeded scale matchings M_s .
 Form degree-ordered partial stages P_s if needed.
 Compute D_s and N_s using (2)–(3).
 Among unused scales, minimize D_s , then maximize N_s ;
 break ties by nominal scale order.
 Append P_s ; update d and mark scale s used.
 Reset used scales after a complete K -stage cycle.
end while
Propagate ancestry by union; return fixed indices I .

degree sum, maximum degree, and index. We then evaluate the prospective spread

$$D_s = \max_i (d_i + \delta_i(P_s)) - \min_i (d_i + \delta_i(P_s)), \quad (2)$$

where $\delta_i(P_s)$ counts occurrences of predecessor i in the retained pairs. Among stages attaining the minimum spread, the constructor favors lower ancestry overlap. For the complete candidate matching M_s , its score is

$$N_s = \frac{1}{|Q_s|} \sum_{(a,b) \in Q_s} \left(1 - \frac{|A_a \cap A_b|}{\max(1, \min(|A_a|, |A_b|))} \right). \quad (3)$$

Here, Q_s contains all pairs in M_s , or 2,048 uniformly indexed pairs for a larger stage. The score approaches zero for identical ancestry and one for disjoint ancestry, using the complete candidate stage even when only a subset fits. Equal scores follow the cyclic nominal scale order, and each scale is used once before a new cycle begins. Algorithm 1 summarizes this selection.

For even widths, every complete matching increments each predecessor’s degree once, while a final subset increments a predecessor at most once. An initially uniform degree vector therefore ends with spread at most one after the complete stages and the final subset. This property follows from disjoint end-points, but native dense random connectivity already balances fan-out and therefore provides an equally relevant control. A degree-preserving ablation is necessary to examine whether the

particular predecessor combinations provide value beyond their degree distribution.

D. Convolutional adaptation

The convolutional adaptation applies the pair constructor to channels while retaining each logic tree’s spatial receptive-field sampler, depth, and weight sharing. The evaluated architecture uses two-channel groups, depth-three trees, and OR pooling, with trainable gate functions shared across spatial positions [4]. First-layer pairing uses channel and threshold semantics, while later layers use the multiscale matching rule to select their channel groups. Thus, the construction changes channel access without replacing convolution with a dense spatial graph.

For this channel representation, ancestry starts with encoded-channel identities and gains a distinct identity label for each subsequent output channel. At flattening, each feature retains its upstream labels and receives a spatial-feature identity. The classifier first uses semantic pairing over the resulting channel-position grid, then the deeper matching rule. These labels prevent channel-support saturation from erasing all distinctions, but remain construction metadata rather than exact pixel-dependence sets.

E. Cost and deployment

Across both families, construction uses packed ancestry bitsets on the CPU before training. Candidate-stage scoring adds offline work and temporary storage, whose construction cost must be distinguished from GPU allocation during training. For a raw dense layer with m gates, the model retains $16m$ trainable gate logits and $2m$ fixed predecessor indices, with no trainable routing state. Export discards the ancestry metadata and retains the selected Boolean functions and indices; physical implementation costs still require synthesis and routing measurements.

IV. EXPERIMENTAL EVALUATION

A. Protocol and main accuracy results

We evaluate MNIST [9], Fashion-MNIST [10], and CIFAR-10 [11] using the saved configurations summarized in Table I. Each attribution pair shares the architecture, encoding, optimizer, budget, split, and seed, with 10% validation data selected using split seed 2027. The archived runs use RTX PRO 6000 Blackwell Max-Q GPUs and PyTorch 2.9.0 with CUDA 13.0, selecting checkpoints by hardened validation accuracy. Tables

TABLE I: Main evaluation configurations. b : thresholds per raw value; K denotes 1,000. Dense widths enumerate gates per layer; convolutional widths enumerate output channels with $k = 32/256$ for S/M.

Dataset/model	Layer widths	Gate functions	b	Updates	Batch
MNIST	$5 \times 1,334 + 1,330$	8K	1	108K	100
Fashion-MNIST	$5 \times 2,666 + 2,670$	16K	3	108K	100
CIFAR-10 S	$4 \times 12K$	48K	3	108K	100
CIFAR-10 M	$4 \times 128K$	512K	3	108K	100
CIFAR-10 L	$5 \times 256K$	1,280K	5	108K	100
Conv. S/M	$(k, 4k, 16k, 32k)$	83,552 / 668,416	3	350K / 200K	128

Dense: Adam, learning rate 0.01, no augmentation. Convolutional: AdamW, learning rate 0.02, decay 0.002, random crops with four-pixel padding and horizontal flips. All use raw gate logits and validation every 2K updates. Convolutional S/M share gate functions across 874,496/6,995,968 spatial gate applications; each classifier uses three dense layers of widths $(1280k, 640k, 320k)$.

TABLE II: Dense held-out accuracy (T, %). Local entries are mean $\pm$ sample SD over seeds 0, 1, 2. U2 gains are percentage points against the paired random baseline.

Model	Gates, K	Random (REP)	V3 (OUR)	U2 (OUR)	Δ U2	Paired 95% CI	Random (R)
MNIST	8	91.27 $\pm$ 0.22	91.91 $\pm$ 0.31	91.94 $\pm$ 0.14	+0.66	[0.02, 1.31]	–
Fashion-MNIST	16	85.20 $\pm$ 0.26	85.91 $\pm$ 0.36	85.72 $\pm$ 0.45	+0.52	[−1.02, 2.06]	–
CIFAR-10 S	48	48.91 $\pm$ 0.35	52.39 $\pm$ 0.30	52.10 $\pm$ 0.63	+3.18	[0.77, 5.60]	51.27 $\pm$ 0.26
CIFAR-10 M	512	54.10 $\pm$ 0.17	58.36 $\pm$ 0.25	58.65 $\pm$ 0.17	+4.56	[3.78, 5.33]	57.39 $\pm$ 0.13
CIFAR-10 L	1,280	55.87 $\pm$ 0.31	61.07 $\pm$ 0.46	60.46 $\pm$ 0.35	+4.59	[3.72, 5.47]	60.78 $\pm$ 0.12

U2 wins all three seed pairs at each coordinate. R: original-paper random baseline and reported spread [1]; no exact-budget published reference is inserted for MNIST or Fashion-MNIST. Reported values are contextual references, not observations from the paired local protocol.

TABLE III: Convolutional CIFAR-10 accuracy and measured training cost. Single-seed test results and three-seed validation replications are separate evidence.

Size	Method	Scope / seeds	Accuracy, %	Time, h	GPU, GiB	Random T (R)
S	Random (REP)	T / 1	57.37	4.98	1.831	60.38
S	U2 (OUR)	T / 1	60.63	4.95	1.831	–
M	Random (REP)	T / 1	69.57	25.47	14.614	71.01
M	U2 (OUR)	T / 1	71.65	35.69	14.614	–
S	Random (REP)	V / 3	59.82 $\pm$ 1.13	4.97	1.831	–
S	U2 (OUR)	V / 3	61.05 $\pm$ 1.04	4.97	1.831	–

R: published convolutional S/M results [4]. Full-S validation uses seeds 0, 1, 2: U2 – random = +1.23 pp, paired 95% CI [−3.36, 5.82], 2/3 wins. Additional S seeds have no held-out test evaluation. Time is the per-run mean; memory is the maximum recorded peak allocation.

distinguish locally reproduced baselines (REP), our methods (OUR), and paper-reported references (R), with V/T denoting validation/test accuracy. Intervals use paired Student- t statistics over training seeds, with sample standard deviations reported alongside means.

Table II compares three paired seeds for random, V3, and U2; V3 uses semantic ordering, butterfly pairing, and ancestry-based swaps. U2 improves dense CIFAR-10 test accuracy by 3.18, 4.56, and 4.59 percentage points at S, M, and L, with positive paired intervals. The MNIST interval also excludes zero, whereas the Fashion-MNIST interval includes zero despite a positive mean across the three seeds. V3 remains more accurate at L, preventing a claim that the unified construction dominates

every dense specialization. Published baselines differ from local results and provide context rather than paired evidence.

The convolutional comparison in Table III retains this distinction. Existing single-seed S/M tests improve by 3.26/2.08 points, respectively. Additional full-S seeds yield 61.05% mean U2 validation accuracy versus 59.82% for random, giving a 1.23-point gain with interval [−3.36, 5.82]. U2 loses on seed 2, exposing variability that the existing single-seed test comparison cannot characterize across independent training runs. These new checkpoints have no held-out test results, so the single-seed test gains do not establish a replicated convolutional improvement.

TABLE IV: CIFAR-10 routing trade-off under LILogic architectures: M is $1 \times 64\text{K}$ gates; L is $2 \times 128\text{K}$ gates. Both use seven input thresholds. Local accuracy is T; R retains the published statistic.

Size	Method	n	Accuracy, %	Params., M	Time, min	GPU, GiB	Accuracy (R)
M	Random (REP)	3	49.01 ± 0.43	1.024	14.62	0.474	49.17 ± 0.14
M	U2 (OUR)	3	52.54 ± 0.30	1.024	14.59	0.474	–
M	Top-32 (REP)	1	57.84	5.120	37.81	8.100	57.28 ± 0.30
L	Random (REP)	3	55.33 ± 0.47	4.096	15.20	1.557	54.76 ± 0.27
L	U2 (OUR)	3	60.19 ± 0.29	4.096	18.47	1.557	–
L	Top-32 (REP)	1	62.03	20.480	350.63	24.861	60.98 ± 0.19

Local protocol: 35K updates, batch 256, Adam at 0.075, random crops with reflected padding and horizontal flips. R: LILogic Net, archived version 2 [3]. U2/random have no trainable routing parameters; Top-32 adds 4.096M/16.384M routing parameters at M/L. The Top-32 resource comparison is descriptive because each learned-routing configuration has one local seed.

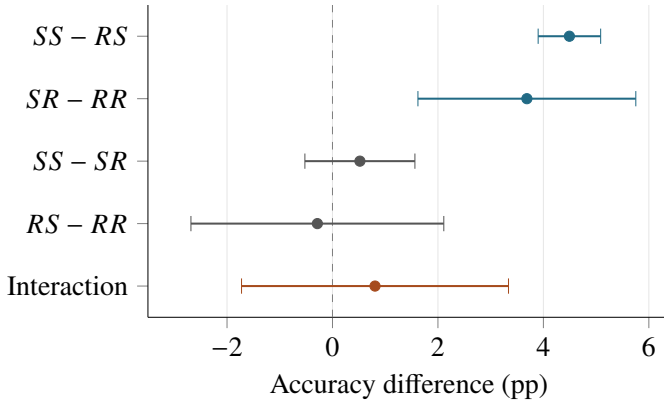


Fig. 2: Dense M factorial effects with paired 95% Student- t intervals ($n = 3$, V). The first two contrasts isolate first-layer structure; the next two isolate deeper structure. Interaction is $(SS - SR) - (RS - RR)$. Intervals are exploratory and unadjusted.

TABLE V: Dense M factorial ablation (20K updates, V). Arm letters indicate first/deeper layers; S is U2 structure and R is a degree-preserving randomization.

Arm	Seeds 0, 1, 2	Mean $\pm$ SD
SS	58.86, 59.48, 59.06	59.13 ± 0.32
SR	58.82, 58.78, 58.24	58.61 ± 0.32
RS	54.26, 55.26, 54.40	54.64 ± 0.54
RR	54.80, 54.48, 55.50	54.93 ± 0.52

All arms use 1.123 GiB peak allocation. Figure 2 shows all five paired contrasts. The first-layer effects and SS-SR win 3/3 seed pairs; RS-RR wins 1/3 and the interaction is positive for 2/3.

B. Training-resource trade-offs

Across matched dense configurations, U2 retains the same peak GPU allocations as random connectivity while adding a separate offline construction step. For U2, construction takes approximately 14.33 seconds at M and 31.22 seconds at L, compared with 41.39 and 113.53 minutes of recorded training. Full-S training averages 4.97 hours for both methods at 1.83

GiB, whereas Full-M U2 records 35.69 hours versus 25.47 for random. Different execution conditions prevent attributing that difference solely to topology, but the observed cost must remain visible.

Table IV compares U2 with learned Top-32 routing under the LILogic architectures and encoding. U2 uses 80% fewer training parameters but loses 5.30/1.84 accuracy points at the two sizes. Top-32 allocates 17.1/16.0 times the peak GPU memory of U2, exposing the resource cost of its additional candidate signals. The Top-32 runs have one local seed, so these descriptive comparisons identify a resource trade-off without establishing an accuracy advantage over learned routing. All memory values are peak PyTorch allocations, and wall times include evaluation overhead.

C. Where does structure help?

To examine the dense gain, we cross structured (S) and randomized (R) first/deeper layers at M with 20K updates. This randomization preserves each predecessor’s degree in each input slot, controlling endpoint reuse while changing which predecessors feed each gate. Full U2 attains the highest mean in Table V, with first-layer effects of 4.49/3.69 points under structured/randomized deeper layers. Both first-layer intervals exclude zero, whereas the additional deeper-layer effect remains uncertain under the same degree-preserving randomization. Figure 2 shows all five contrasts; the deeper-layer gain is 0.52 points with interval $[-0.52, 1.56]$. All five contrasts are exploratory and unadjusted.

The corresponding convolutional body/classifier study finds a 2.14-point body effect with a fixed reference-U2 classifier, interval $[0.67, 3.61]$. That effect remains conditional because the fixed classifier uses reference-body ancestry; separate controls do not establish an incremental benefit from ancestry-based selection over nominal stages. Adapted WARP preserves a dense U2 advantage but gives an inconclusive convolutional effect; both adapted variants trail the matched raw parameterization. These outcomes bound the mechanism and compatibility claims.

D. Transfer boundary

Frozen CIFAR-10.1 [12] evaluation gives dense M a 3.73-point U2 gain over random, with paired 95% interval

[2.80, 4.67] across three seeds. Convolutional S/M changes are -0.05/+1.95 points with one seed each, and CIFAR-100 pilots do not establish a U2 benefit. These outcomes separate transfer of the construction from evidence of a consistent accuracy gain.

V. CONCLUSION AND DISCUSSION

We presented LogicConnect, a unified construction that organizes connectivity before gate-function training in dense and convolutional LGNs. Semantic pairing preserves the meaning of image inputs, while deeper matching stages provide deterministic multiscale connections with controlled predecessor reuse. The convolutional adapter applies the same construction to channel groups and the classifier while preserving spatial sampling. The trained model retains ordinary fixed indices and gate functions, with no additional routing parameters or inference operators.

The dense results establish the value of this design choice under the evaluated budgets. U2 improves all three CIFAR-10 sizes over matched random graphs, and the factorial study identifies first-layer structure as the dominant contributor at dense M. However, the available controls do not isolate ancestry novelty as the source of improvement, and native random connectivity already balances fan-out. The additional contribution of deeper structure remains uncertain, even though the full construction attains the highest mean in the dense factorial study. The method should therefore be understood as a tested construction, rather than a proven coverage-maximization principle.

The convolutional evidence requires a narrower interpretation because architectural compatibility and replicated accuracy gains answer different experimental questions. Single-seed held-out improvements demonstrate feasible application to both S and M, but full-S validation replications do not establish a consistent gain. The transfer experiments reinforce the need to preserve this distinction when moving beyond the original training distribution or class count. A shared construction reduces the number of design rules, while its empirical value remains specific to the evaluated architecture and workload.

For deployment-oriented design, U2 offers a measured alternative when training memory matters and fixed architectural budgets are required. Learned routing can remain more accurate, while unchanged logical cost leaves physical area, delay, and energy dependent on the eventual implementation. Additional held-out convolutional replications and matched synthesis are therefore needed to establish the corresponding accuracy and physical implementation trade-offs. The supported design insight is that organizing accessible signals before optimization can improve the accuracy obtained from a fixed logic graph.

REFERENCES

- [1] F. Petersen, C. Borgelt, H. Kuehne, and O. Deussen, “Deep differentiable logic gate networks,” in *Advances in Neural Information Processing Systems*, vol. 35, 2022.
- [2] W. Mommen, L. Keuninckx, M. Hartmann, and P. Wambacq, “A method for optimizing connections in differentiable logic gate networks,” arXiv:2507.06173, 2025.
- [3] K. Fojcik, R. Zioma, and J. Armitis, “LILogic Net: Compact logic gate networks with learnable connectivity for efficient hardware deployment,” arXiv:2511.12340v2, 2026.
- [4] F. Petersen, H. Kuehne, C. Borgelt, J. Welzel, and S. Ermon, “Convolutional differentiable logic gate networks,” in *Advances in Neural Information Processing Systems*, vol. 37, 2024.
- [5] S. Bührer, A. Plesner, T. Aczel, and R. Wattenhofer, “BitLogic: A framework for gradient-based LUT-native neural networks,” *Transactions on Machine Learning Research*, 2026.
- [6] L. Rüttgers, T. Aczel, A. Plesner, and R. Wattenhofer, “Light differentiable logic gate networks,” arXiv:2510.03250, 2025.
- [7] L. Gerlach, T. Gerlach, L. Våge, E. Kauffman, and I. Ojalvo, “WARP logic neural networks,” arXiv:2602.03527, 2026.
- [8] S. Lee, J. Sun, and J. Chung, “Two-stage unit tying for simplifying differentiable logic gate networks,” in *Proceedings of the International Conference on Machine Learning*, ser. Proceedings of Machine Learning Research, vol. 306, 2026.
- [9] Y. LeCun, L. Bottou, Y. Bengio, and P. Haffner, “Gradient-based learning applied to document recognition,” *Proceedings of the IEEE*, vol. 86, no. 11, pp. 2278–2324, 1998.
- [10] H. Xiao, K. Rasul, and R. Vollgraf, “Fashion-MNIST: A novel image dataset for benchmarking machine learning algorithms,” arXiv:1708.07747, 2017.
- [11] A. Krizhevsky, “Learning multiple layers of features from tiny images,” University of Toronto, Tech. Rep., 2009.
- [12] B. Recht, R. Roelofs, L. Schmidt, and V. Shankar, “Do CIFAR-10 classifiers generalize to CIFAR-10?” arXiv:1806.00451, 2018.